Lab 02 – Prompt Development History

Chronological transcript of the coding, debugging, and dashboard-development prompts used in this lab session (reservoir stage analysis).

Step 1 — Initial Script: Time Axis & Finite-Difference Derivatives

Type: Coding request (task specification)

PROMPT

> Please solve the following reservoir laboratory problem based on the provided dataset. Please create a python script:
>
> Task Instructions:
> 1. Time Axis Conversion: Convert the timestamps into a numeric time axis representing elapsed hours from the first reading (t = 0 h) using a step size of dt = 0.25 h. Keep this time origin and units (hours, meters) consistent throughout.
> 2. First Derivative (dh/dt): Calculate the first derivative for all 96 points using: Forward difference for the first data point · Central difference for all intermediate data points · Backward difference for the last data point
> 3. Second Derivative (d2h/dt2): Calculate the second derivative values across the dataset.
> 4. Analysis and Deliverables: State the time of maximum dh/dt and its numerical value in meters per hour (m/h). Provide a one-sentence physical interpretation of what the second derivative indicates about the inflow (not just noting where it changes sign).

RESULT / RESPONSE SUMMARY

- Loaded the uploaded Excel sensor log (288 readings, 15-min interval) — noted the actual record was 288 points, not 96 as stated in the task, and proceeded with the full record for time-axis consistency.
- Built t = index × 0.25 h; computed dh/dt with forward/central/backward finite differences and d^2h/dt^2 with a 3-point central formula (one-sided at the endpoints).
- Found max dh/dt = 0.96 m/h at t = 29.25 h (Reading #118, Jul 22 05:15) and confirmed it sits inside a real, sustained rise (not a noise spike).
- Interpreted d^2h/dt^2: positive = accelerating inflow, negative = waning inflow rate.
- Delivered reservoir_analysis.py and a CSV of (t, h, dh/dt, d^2h/dt^2).

Step 2 — Nonlinear Curve Fitting (Levenberg–Marquardt)

Type: Coding request / parameter & model specification

PROMPT

Now add this to the python script. Just base on the code you provided earlier. Please perform nonlinear curve fitting and statistical evaluation on the provided reservoir dataset using the Levenberg-Marquardt method:

1. Model Selection and Defense: choose the best fitting model among Four-parameter logistic, Gompertz, Sum of two logistics, or Cubic polynomial; defend the choice in exactly two sentences before running the fit.
2. Initial Guesses (p0): formulate realistic p0 values based directly on the dataset and explain the source of each number — no default values of ones.
3. Model Fitting: scipy.optimize.curve_fit with method='lm' and maxfev=20000, using t in hours and h in meters.
4. Statistical Computation (in exact order): popt, residuals $e_i = h_i - h_{hat}$, SSE, SST, R^2, standard error of the estimate $s = \sqrt{SSE/(n-p)}$, parameter SE = sqrt(diag(pcov)), t-statistics = popt/se, two-tailed p-values = 2*(1–stats.t.cdf(|t|, n–p)).
5. Reporting: a summary table (parameter, SE, t-stat, p-value) with results rounded to match sensor precision (nearest centimeter).

RESULT / RESPONSE SUMMARY

- Inspected the data shape (baseline → rapid rise → peak → partial recession → new plateau) and selected **Sum of two logistics** over the other three candidates, since the record has two sigmoid transitions and two flat asymptotes.
- Derived p0 directly from the data: baseline = mean of first 20 readings, amplitudes from peak vs. baseline/final plateau, inflection times from where dh/dt is max/min, rates from the logistic slope-at-inflection relation $k = 4 \cdot slope/A$.
- Fitted with curve_fit(method='lm', maxfev=20000) and computed the full statistical sequence exactly in the order requested.
- Result: $R^2 = 0.9991$, s = 0.080 m, all seven parameters significant at $p < 0.001$.

Step 3 — Self-Audit of the Fitting/Statistics Code

Type: Debugging / verification request

PROMPT

Please review and audit my current Python script against the required checklist to verify that all necessary fitting steps and statistical metrics have been properly implemented and computed. Verification Checklist: (1) Levenberg-Marquardt fitting explicitly executed with method='lm'; (2) fitted parameters (popt) correctly extracted; (3) residuals computed as resid = h – model(t, *popt); (4) all required statistical metrics present with correct formulas — SSE, SST, R^2, standard error of the estimate, parameter standard errors from the covariance matrix, t-statistics, and two-tailed p-values with n–p degrees of freedom. Inspect the code, point out any missing formulas, incorrect degrees of freedom, or variable-handling errors, and provide exact fix lines if anything is wrong.

RESULT / RESPONSE SUMMARY

- Audited the script line-by-line against all 7 checklist items and confirmed full compliance: correct method='lm' call, correct residual formula, correct SSE/SST/R^2/s formulas, matching n–p = 281 degrees of freedom used consistently in both the s and p-value calculations.
- No missing formulas or errors found — no code changes were required at this step.

Step 4 — Residual Diagnostics & Area Integration

Type: Coding request / theoretical interpretation

PROMPT

1. Residual Visualization: generate residuals-vs-time and residuals-vs-fitted-values scatter plots, each with a horizontal zero reference line.
2. Residual Analysis (written): Is the scatter centered on zero or drifting? Long runs of the same sign? Does spread widen as level rises? Is the largest residual larger than the logger's 1 cm resolution, and by how much? Conclude whether the fit is acceptable or a failed fit despite the R^2 value.
3. Area Integration: integrate h_hat(t) with scipy.integrate.quad over [t0, tn]; report the area and quad's absolute error estimate; state units explicitly as meter-hours (m·h), not a volume.
4. Cross-Check: cross-check with np.trapezoid on the raw readings; explain in one sentence why the two areas differ; explain in 1–2 sentences what the integrated value means (and does not mean) to a flood-control office.

RESULT / RESPONSE SUMMARY

- Added residuals-vs-time and residuals-vs-fitted plots (matplotlib) with zero reference lines.
- Diagnosed a clear, structured “double-hump” pattern in the residuals (not random noise): longest same-sign run = 54 points (13.5 h), max |residual| = 0.221 m (≈22× the 1 cm sensor resolution) — concluded the fit is a **failed local-shape fit despite R^2 = 0.9991**.
- Computed area via quad = 1260.97 m·h (abs. error ≈ 5.09×10^{-7} m·h) and cross-checked with np.trapezoid = 1261.00 m·h (difference –0.03 m·h, –0.002%).
- Explained the physical meaning of the m·h integral for a flood-control office (event severity/duration proxy) and explicitly noted it is not a water volume.

Step 5 — First HTML Dashboard Build

Type: Dashboard generation (coding)

PROMPT

Please update the Python script so that running it automatically generates a standalone, beautifully structured HTML dashboard file (e.g., dashboard.html) that opens directly upon double-clicking with no build step or server required.

Requirements: clean modern styling in structured cards and tables; responsive, sharp, self-contained charts; a header (always visible) with the raw stage time series; a Derivatives tab with dh/dt and d^2h/dt^2 and the max dh/dt point annotated; a Model Fit & Statistical Evaluation tab with the fitted-curve overlay, equation, residuals-vs-time plot with written interpretation, a parameter summary table, and goodness-of-fit metric cards (SSE, SST, R^2, s with n and p); and an Integration & Area tab with the shaded fitted-curve area, the quad result with its error estimate, units, and the trapz cross-check.

RESULT / RESPONSE SUMMARY

- No internet access was available to bundle Plotly, so charts were built as hand-written, responsive inline SVG (fully offline-safe) rather than relying on a CDN library.
- Built a reusable SVGChart helper (axes, gridlines, lines, scatter, shaded area, zero reference lines, annotated markers) and generated all required charts and cards across a header + 3 tabs.
- Validated all SVG blocks as well-formed XML, checked for NaNs, and rendered screenshots at desktop and mobile widths (via wkhtmltoimage) to confirm layout quality before delivering dashboard.html alongside the updated script.

Step 6 — Dashboard Audit & Debugging Against Full Spec

Type: Debugging / verification request

PROMPT

Please review, debug, and audit the Python script and its generated HTML dashboard against the final required presentation specifications below. Make sure that all listed components are explicitly present and correctly displayed in the final HTML file: (1) Main Dashboard/Header: raw water level vs. time plot + fitted model/curve overlay; (2) Derivatives tab: dh/dt, d^2h/dt^2, max dh/dt clearly marked; (3) Fitted Model tab: raw+fit overlay, explicit equation, parameter summary; (4) Residuals section: residual-vs-time AND residual-vs-fitted plots, both with a visible zero line, plus written interpretation; (5) Statistics section: parameter table (SE, t, p) and goodness-of-fit metrics (SSE, SST, R^2, s, dof = n–p); (6) Area tab: shaded fitted-curve area, quad result + error, trapz cross-check, explicit units. Check for missing sections, broken tab switches, missing plot traces, or misaligned statistics, and provide the complete, debugged, finalized script.

RESULT / RESPONSE SUMMARY

- Audited the existing dashboard against a 20-point programmatic + visual checklist and found two gaps: (1) the header only showed the raw log, missing the required fitted-curve overlay; (2) the residuals-vs-fitted-water-level chart existed only as a standalone PNG from an earlier step and was never embedded in the HTML dashboard.
- Fixed both: added h_hat as a second (dashed) line with a legend in the header chart, and added a new embedded SVG chart for residuals-vs-fitted with its own zero reference line.
- Re-validated all SVG blocks, re-ran the full 20-item checklist (all passing), and re-rendered all three tabs on desktop and mobile widths to confirm the fix before delivering the final script.